



Vigil: Decentralized Liquidation Prediction

The Problem

A user deposits \$100,000 into Aave as collateral and borrows \$70,000. Market drops 20% overnight. Their health factor crosses 1.0. They get liquidated.

No warning. No alert. Just a 10% penalty: \$7,000 gone.

Meanwhile, a liquidator with a private prediction bot saw this coming 3 hours ago. They positioned themselves, executed the liquidation, and pocketed the bonus.

\$2.4 billion was liquidated across DeFi in 2024. The borrowers who lost had no warning. The liquidators who won had proprietary prediction systems.

This intelligence exists. It's just locked inside private bots.

The Solution

Vigil is a Bittensor subnet where miners compete to predict DeFi liquidations before they happen.

- Miners analyze on-chain positions and predict which will be liquidated
- Validators verify predictions against actual liquidation events
- Accurate predictions earn TAO

The result: Decentralized liquidation intelligence that anyone can access.



1. Mechanism Design

1.1 Overview

EVERY EPOCH (1 hour):

1. T=0:00 Validators snapshot at-risk positions ($1.0 < HF < 1.5$)
2. T=0:00 Miners receive position list
3. T=0:10 Prediction window closes (commit-reveal complete)
4. T=0:10 Observation window STARTS
5. T=6:10 Observation window ends (6 hours after predictions locked)
6. T=6:10 Validators match predictions to actual liquidations
7. Miners scored on accuracy (rolling 24-hour aggregation)
8. TAO distributed to top performers

Key timing details: - Only positions with $HF > 1.0$ are included (already-liquidatable positions are excluded) - Observation window starts AFTER prediction window closes - Liquidations during the 10-minute prediction window are excluded from scoring - This ensures miners always have time to analyze before being judged

1.2 Emission Split

Recipient	Share
Miners	41%
Validators	41%
Subnet Creator	18%

Standard Bittensor allocation.

1.3 Scoring Formula

Miners are scored on four dimensions:



Dimension	Weight	What It Measures
Precision	40%	Of predictions made, how many were correct?
Recall	30%	Of liquidations that happened, how many did you predict?
Lead Time	20%	How early was the prediction? (requires 50%+ precision to activate)
Calibration	10%	Does stated confidence match actual accuracy? (3-bin system)

Why these weights: - Precision (40%): Prevents "predict everything" gaming - Recall (30%): Rewards catching actual liquidations - Lead Time (20%): Earlier predictions are more valuable - Calibration (10%): Lower weight due to measurement noise

Final score (rolling 24-hour window):

```
base_score = (precision × 0.40) + (recall × 0.30)

IF precision >= 0.50:
    lead_time_bonus = min(avg_lead_time_hours / 6, 2.0)
ELSE:
    lead_time_bonus = 1.0 # No bonus for low-precision miners

calibration_score = 1 - brier_score_binned # 3-bin system

final_score = base_score + (lead_time_bonus × 0.20) + (calibration_score × 0.10)
```

Lead time bonus requires 50%+ precision. This prevents gaming where miners make wild early guesses hoping for the lead time multiplier.

1.4 Danger Zone Partial Credit

Problem: A miner predicts liquidation, but the borrower adds collateral and saves the position. This seems like a false positive, but the prediction was reasonable.

Solution: Partial credit for positions that entered the "danger zone."



OUTCOME CLASSIFICATION:

1. TRUE POSITIVE (full credit)
 - Predicted liquidation
 - Position was liquidated
2. DANGER ZONE HIT (partial credit: 0.5)
 - Predicted liquidation
 - Position hit HF < 1.02 but was rescued
 - Prediction was reasonable, borrower intervened
3. FALSE POSITIVE (no credit)
 - Predicted liquidation
 - Position never approached danger (HF stayed > 1.1)
4. FALSE NEGATIVE (penalty)
 - Did not predict
 - Position was liquidated

This provides more data points and fairer scoring, especially during low-volume periods.

1.5 Rolling 24-Hour Aggregation

Problem: With only 5-10 liquidations per day, scoring per epoch is statistically noisy.

Solution: Aggregate scores over a rolling 24-hour window.

```
miner_score = aggregate(  
    epochs[-24:], # Last 24 epochs (24 hours)  
    weights=time_decay # Recent epochs weighted slightly higher  
)
```

This provides 50-200+ data points for scoring, making results statistically meaningful.

1.6 Calibration: 3-Bin System

Problem: Measuring "70% confidence = 70% accuracy" requires many predictions at exactly 70% confidence. Not realistic.

Solution: Group confidence into 3 bins.



Bin	Confidence Range	Expected Accuracy
Low	0-40%	Should be wrong more than right
Medium	40-70%	Should be right roughly half the time
High	70-100%	Should be right most of the time

Calibration score measures whether each bin performs as expected, aggregated over 7 days.

1.7 Anti-Gaming Summary

Attack	Defense
Predict everything	Precision penalty (40% weight)
Predict nothing	Recall penalty (30% weight)
Wild early guesses	Lead time bonus requires 50%+ precision
Copy other miners	Commit-reveal mechanism
Predict only obvious ($HF \approx 1.0$)	Minimum 1-hour lead time required
Game confidence scores	3-bin system with 7-day rolling window

2. Miner Design

2.1 Task

Miners monitor DeFi lending positions and predict liquidations.

What miners do: 1. Index positions from Aave V3 2. Track health factors, collateral values, debt amounts 3. Analyze market conditions and price trends 4. Predict which positions will liquidate and when

2.2 What Makes Liquidation Prediction Hard

This isn't just price prediction. Miners must understand:



Factor	Why It Matters
Borrower behavior	Will they add collateral? Repay debt?
Cascading effects	One liquidation can trigger others
Liquidator dynamics	When gas is high, liquidators wait
Oracle delays	Chainlink heartbeat means price updates aren't instant
Protocol mechanics	Liquidation thresholds vary by asset and protocol

A miner who only predicts price will underperform one who understands these factors.

2.3 Input

Validators provide position snapshots:

```
{
  "epoch": "2026-02-05T14:00:00Z",
  "positions": [
    {
      "id": "aave_v3_eth_0x1234",
      "protocol": "aave_v3",
      "chain": "ethereum",
      "user": "0x1234...abcd",
      "health_factor": "1.15",
      "collateral_usd": "100000",
      "debt_usd": "75000",
      "collateral_assets": ["WETH"],
      "debt_assets": ["USDC"]
    }
  ],
  "market_data": {
    "eth_price_usd": 2500,
    "eth_24h_change": -0.05,
    "gas_price_gwei": 30
  }
}
```

2.4 Output

Miners submit predictions:



```
{
  "miner_id": "5F3sa...",
  "epoch": "2026-02-05T14:00:00Z",
  "predictions": [
    {
      "position_id": "aave_v3_eth_0x1234",
      "will_liquidate": true,
      "time_window_hours": [2, 6],
      "confidence": 0.75
    },
    {
      "position_id": "aave_v3_eth_0x5678",
      "will_liquidate": false,
      "confidence": 0.90
    }
  ],
  "commitment_hash": "0xabc..."
}
```

2.5 Miner Edge

What separates good miners from bad:

Edge	Description
Better price models	More accurate volatility forecasting
Historical analysis	Learning from past liquidation patterns
Protocol expertise	Understanding Aave-specific mechanics
Real-time data	Faster position monitoring
Whale tracking	Monitoring large wallet movements

3. Validator Design

3.1 Task

Validators verify miner predictions against on-chain reality.



What validators do: 1. Snapshot positions at epoch start 2. Collect and verify miner commitments 3. Monitor for liquidation events 4. Score miners based on outcomes 5. Submit scores to Bittensor consensus

3.2 Verification

Liquidations are on-chain facts:

```
For each prediction:
  1. Query: Did position X get liquidated?
     → Check for LiquidationCall events
  2. If yes: When? (block timestamp)
     → If liquidation occurred BEFORE observation window started, EXCLUDE from scoring
  3. If no: Did position enter danger zone? (HF < 1.02)
  4. Compare to miner's prediction
  5. Classify outcome and score
```

Exclusion Rules:

Scenario	Action	Rationale
Position HF < 1.0 at snapshot	Exclude from prediction set	Already liquidatable, not predictable
Liquidation during 10-min prediction window	Exclude from scoring	Miners had no chance to predict
Position removed/repaid before observation	Exclude from scoring	No longer a valid prediction target

No subjectivity. Validators verify facts, not judge quality.



3.3 Scoring Logic

```
def score_miner(predictions, actual_events, position_snapshots):
    outcomes = []

    for pred in predictions:
        liquidation = find_liquidation(pred.position_id, actual_events)
        min_hf = get_minimum_health_factor(pred.position_id, observation_window)

        if pred.will_liquidate:
            if liquidation:
                # True positive
                outcomes.append(('TP', 1.0, liquidation.timestamp - pred.timestamp))
            elif min_hf < 1.02:
                # Danger zone hit - partial credit
                outcomes.append(('DZ', 0.5, None))
            else:
                # False positive
                outcomes.append(('FP', 0.0, None))
        else:
            if liquidation:
                # False negative
                outcomes.append(('FN', -0.5, None))
            else:
                # True negative
                outcomes.append(('TN', 0.1, None))

    return calculate_final_score(outcomes)
```

3.4 Evaluation Cadence

Parameter	Value	Rationale
Epoch length	1 hour	Balance freshness with gas costs
Prediction window	10 minutes	Time for miners to analyze and submit
Observation window	6 hours	Starts AFTER prediction window closes
Total epoch duration	6h 10m	Prediction (10m) + Observation (6h)
Score aggregation	Rolling 24 hours	Statistical significance
Calibration window	Rolling 7 days	Enough data for bin accuracy

Note: Overlapping epochs are allowed. A new epoch starts every hour, even while previous observation windows are still open. This means ~6 concurrent observation windows at any time.



3.5 Validator Incentive Alignment

Why do validators score honestly?

Three mechanisms ensure validator integrity:

1. Yuma Consensus (Bittensor Protocol)

Validators are weighted by agreement with other validators. Outliers are penalized.

```
Validator A scores: [0.8, 0.2, 0.5] ← agrees with consensus
Validator B scores: [0.7, 0.3, 0.5] ← agrees with consensus
Validator C scores: [0.1, 0.9, 0.2] ← outlier, discounted

Result: Validator C earns less TAO
```

2. Objective Ground Truth (Vigil-Specific)

Unlike subjective tasks, Vigil has provable ground truth:

Validator Claim	On-Chain Reality	Consequence
"No liquidation"	LiquidationCall tx exists	Provably wrong
"HF never hit 1.02"	Archive node shows HF = 1.01	Provably wrong

Validators can't argue about facts. Either the liquidation happened or it didn't. This makes honest scoring the only viable strategy.

3. Compute Cost vs. Reward

Validators must: - Run archive node queries (real cost) - Verify danger zone hits (compute work) - Aggregate miner scores (processing)

Random scoring would be detected by Yuma Consensus. The only way to earn the 41% validator emission is to do the verification work correctly.

Collusion Resistance:

- Miner-validator collusion requires 51%+ of validator stake
- Liquidations are public on-chain events — can't fake them
- Any participant can verify scores independently



4. Why Bittensor

4.1 Proof of Intelligence

Predicting liquidations requires genuine intelligence:

- Analyzing market conditions
- Anticipating borrower behavior
- Timing predictions accurately
- Understanding protocol mechanics

Random predictions have near-zero precision. You can't fake this.

4.2 Objective Verification

Unlike subjective tasks, liquidations are **on-chain facts**:

Data Point	Source	Verifiable
Amount	Event logs	Yes
Health factor	Contract state	Yes
Liquidation happened	Transaction hash	Yes
Timestamp	Block number	Yes

No ambiguity. No disputes. Validators verify facts, not opinions.

4.3 Differentiation from SN10 (Sturdy)

Aspect	SN10 (Sturdy)	Vigil
Goal	Maximize yield (offense)	Predict risk (defense)
Task type	Continuous optimization	Discrete event prediction
Output	Allocation percentages	Binary predictions + timing
Verification	Performance over weeks	Outcome within hours
User	Yield seekers	Risk managers, borrowers



Sturdy helps you make money. Vigil helps you not lose money.

5. Business Logic

5.1 Who Pays for Vigil

Customer	Product	Why They Pay
Analytics platforms	Data feed	Content for their users
Borrowers	Alert subscription	Avoid liquidation penalty (5-15% savings)
Liquidators	Prediction API	Better timing = more profit
Protocols	Risk dashboard	Monitor systemic risk in real-time

5.2 Why Liquidators Use Vigil (Not Just Self-Liquidate)

Question: If miners can predict liquidations, why don't they just liquidate themselves?

Answer: Different skills, different capital requirements.

Prediction	Liquidation
Requires: Analysis skills	Requires: Capital (\$100K+)
Requires: Data infrastructure	Requires: MEV infrastructure
Risk: Lost TAO emissions	Risk: Failed transactions, gas wars
Reward: Steady TAO income	Reward: Variable liquidation profits

Vigil commoditizes prediction. Liquidators compete on execution.

Miners who predict well but can't execute now have a way to monetize their intelligence.



5.3 Market Size

Metric	Value
DeFi lending TVL	\$30B+
Annual liquidation volume	\$2.4B (2024)
Liquidation penalty	5-15%
Liquidator profits	~\$500M annually

Growing market. As DeFi lending grows, so does liquidation volume.

5.4 Revenue Model

Stream	Description	Timeline
TAO emissions	Subnet rewards	Day 1
Borrower alerts	Push notifications for at-risk positions	Month 3+
Liquidator API	Premium prediction feeds	Month 3+
Protocol dashboards	B2B risk monitoring	Month 6+

6. Competitive Landscape

6.1 Within Bittensor

Subnet	Focus	Differentiation
SN10 (Sturdy)	Yield optimization	Offense vs defense
None	Liquidation prediction	Novel: first mover



6.2 Outside Bittensor

Competitor	What They Do	Vigil's Edge
Chaos Labs	Risk simulation	Historical, not predictive
DeFi Saver	Automated protection	Reactive, not predictive
Gauntlet	Risk modeling	B2B only, not real-time, centralized
Liquidation bots	Private prediction	Proprietary, not decentralized

Gap: No decentralized, predictive liquidation intelligence exists.

7. Go-To-Market

7.1 Bootstrap Strategy (Pre-Launch)

The Cold-Start Problem:

- Miners won't join without TAO rewards
- Validators won't join without miners to score
- Users won't pay without quality predictions

Solution: Two-pronged bootstrap

A) Technical Bootstrap: Historical Replay Rewards

Before mainnet has real liquidations, miners earn TAO by predicting historical data:

1. Take mainnet position snapshots from 30 days ago
2. Miners predict based on that state
3. Validators score against known outcomes
4. TAO distributed to accurate miners

This solves cold-start immediately — miners can earn from day one by proving skill on historical data. As real liquidations occur, the system transitions to live predictions.

B) Business Bootstrap: Anchor Partnerships



Pre-launch partnerships to prove demand:

Target Partner	Value Proposition	Status
Liquidation bots (Instadapp, DeFi Saver)	Better timing signals → more profit	Target
Lending protocols (Aave, Compound)	Risk monitoring dashboard	Target
DeFi aggregators (DeBank, Zapper)	User protection alerts	Target

Goal: 2-3 signed LOIs before mainnet launch.

C) Early Miner/Validator Incentives

Incentive	Details
First 50 miners	Featured in launch announcement
First 10 validators	Direct onboarding support
Historical replay leaderboard	Top performers highlighted pre-mainnet

No inflated token rewards — we attract quality participants through reputation and early mover advantage, not unsustainable emissions.

7.2 Phase 1: Launch (Month 1-2)

- Deploy subnet on mainnet
- Transition from historical replay to live predictions
- Reference miner implementation (open source)
- Basic alert bot (Discord/Telegram)
- Documentation and onboarding guides

7.3 Phase 2: Adoption (Month 3-6)

- Borrower dashboard
- Liquidator integrations
- Premium API tiers
- Protocol partnerships (Aave governance proposal)



7.4 Phase 3: Scale (Month 6+)

- Advanced analytics products
- Multi-chain (Arbitrum, Base, Optimism)
- Multi-protocol (Compound, Morpho, Spark)
- Protection products built by third parties

8. Round II Implementation

8.1 Scope

Component	Included	Notes
Position monitoring	Yes	Aave V3 + Compound V3 + Morpho on Ethereum
Miner predictions	Yes	Commit-reveal mechanism
Validator scoring	Yes	Precision/recall/lead time
Danger zone credit	Yes	Partial credit for near-misses
Rolling aggregation	Yes	24-hour window
On-chain warnings	Yes	Event emissions
Multi-protocol	Yes	Aave V3 + Compound V3 + Morpho (~65% of lending market)
Multi-chain	No	Ethereum only

8.2 Protocol Adapter Architecture

The system is built on a protocol-agnostic adapter interface:



```
class ProtocolAdapter:
    name: str
    prediction_window: timedelta    # Protocol-specific (6h, 12h, 24h)
    risk_threshold: float          # When position is "at risk"

    def get_at_risk_positions(self) -> List[Position]
    def get_risk_score(self, position) -> float        # Normalized 0-1
    def get_liquidation_events(self, start, end) -> List[Event]
    def is_danger_zone(self, position) -> bool
```

Round II Adapters:

Protocol	Window	Risk Metric	Coverage
Aave V3	6h	Health Factor < 1	~\$68B TVL
Compound V3	6h	Liquidity < 0	~\$3B TVL
Morpho	6h	Health Factor < 1	~\$13B TVL

Future Adapters (Post-Hackathon):

Protocol	Window	Prediction Target
MakerDAO	24h	Auction start (not completion)

Maker uses auction-based liquidations that take hours to complete. Our architecture supports variable prediction windows, so Maker would use a 24-hour window predicting auction initiation rather than completion. This is documented but not implemented in Round II.

8.3 Historical Replay Mode

Problem: Testnet may have few or no real liquidations.

Solution: Use historical mainnet data for testing.

HISTORICAL REPLAY:

1. Take mainnet position snapshots from 30 days ago
2. Miners predict based on that state
3. Validators score against what actually happened (known ground truth)
4. Proves mechanism works without needing live liquidations

This demonstrates mechanism correctness before mainnet deployment.



8.4 Timeline

Week	Deliverable
1	Adapter interface, position indexing, Aave V3 adapter
2	Compound V3 adapter, Morpho adapter
3	Validator scoring (protocol-agnostic), danger zone logic, rolling aggregation
4	Integration testing with historical replay, documentation, demo

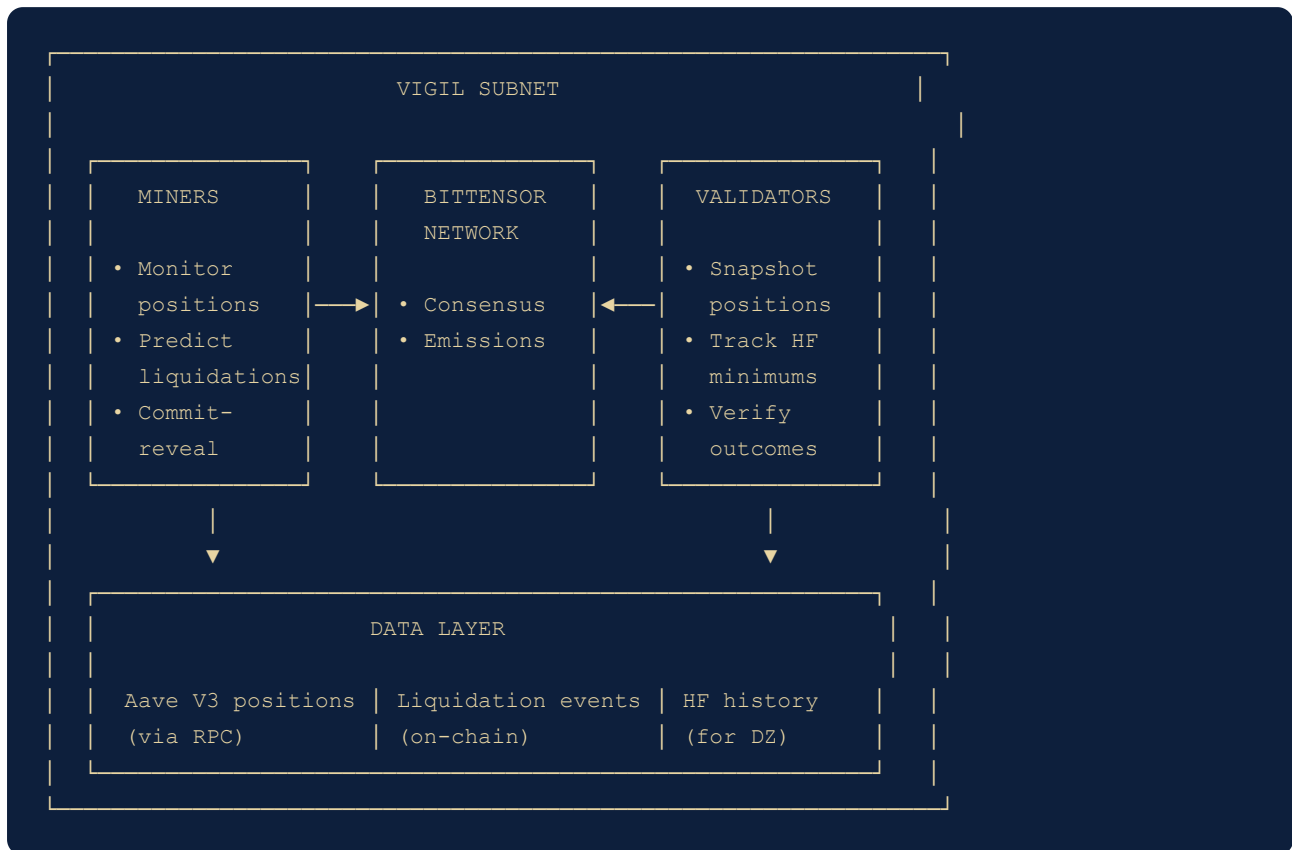
Feature Flags: Each protocol adapter behind a flag. If Morpho or Compound is unstable, disable and ship with working adapters only.

8.5 Success Criteria

- Miners can submit predictions via commit-reveal
 - Validators score against actual/historical liquidations
 - Danger zone partial credit works correctly
 - Rolling 24-hour aggregation produces stable scores
 - TAO flows to accurate miners
-



9. Technical Architecture



10. Risk Factors

Risk	Likelihood	Impact	Mitigation
Few miners at launch	Medium	Medium	Reference implementation + documentation
Low liquidation volume	Medium	Medium	Danger zone credit + 24-hour aggregation
Miner collusion	Low	Medium	Commit-reveal + Yuma Consensus
Oracle manipulation	Low	High	Multiple price source verification
Testnet has no liquidations	High	High	Historical replay mode



Summary

Vigil brings decentralized intelligence to DeFi liquidations.

The problem: \$2.4B liquidated with no warning. Prediction intelligence is locked in private bots.

The solution: Miners compete to predict liquidations. Validators verify against on-chain facts. Best predictors earn TAO.

Key mechanisms: - 3-bin calibration system for noisy data - Danger zone partial credit for fair scoring - Lead time bonus with precision threshold to prevent gaming - Rolling 24-hour scoring for statistical significance

Why Bittensor: - Decentralized (no single entity controls predictions) - Objective verification (liquidations are on-chain facts) - Proof of intelligence (prediction is genuinely hard)

Why now: - Clear path to paying customers - DeFi lending is growing - Novel use case: no Bittensor subnet does this

Round II: Working testnet with historical replay, Aave V3 + Compound V3 + Morpho predictions, and complete scoring mechanism.

Links

Resource	URL
Pitch Deck	https://vigil-pitch-deck.vercel.app
Twitter	@faruukku